Software Testing
42036101
Assignment 1

Group 10

Group leader：冯俊财 (Student ID: 2353924)
Group members：谢雨霏 (Student ID: 2354264)
于伊莲 (Student ID: 2352993)
金子涵 (Student ID: 2351442)
纪鹏 (Student ID: 2351869)

Table of Contents

1. Introduction & System Overview 3
2. Manual Test Case Design 3
2.1 Methodology 3
2.2 convert_number_to_words (Python) 4
2.3 dayjs (JavaScript) 8
3. AI-based Test Generation Tool 12
3.1 Tool Architecture 12
3.2 Prompt Design 13
3.3 Supported LLM Backends 14
3.4 Three Core Design Features 14
4. Experimental Analysis 15
4.1 Experimental Overview 15
4.2 False Negative Analysis 15
4.3 False Positive Analysis 17
4.4 Model Comparison Analysis 18
4.5 Language Difference Analysis 19
4.6 Root Cause Analysis 19
4.7 Improvement Insights 20
5. Project Report 21
5.1 Comparison with Traditional Non-AI White-box Testing Technologies 21
5.2 AI Limitations Encountered in Practice and Improvement Process 24
5.3 Experimental Data Comparison 28
5.4 Project Summary and Outlook 28
6. Appendix 31
Appendix A: Full Prompt Templates 31
Appendix B: Requirements Document 36
Appendix C: Experiment Output Table 39

1. Introduction & System Overview

This experiment selects two programs as the systems under test: convert_number_to_words.py, a Python pure-functional module that converts integers into English words across three numbering s ystems (short, long, and Indian); and Day.js index.js, the core module of a lightweight JavaScript date-time library that exposes a chain-call object-oriented API. Both programs were chosen for the ir representativeness — the former features recursive logic and enumeration-driven branching in a pure-functional style, while the latter involves complex OOP chain calls, timezone handling, and f loating-point arithmetic, making them complementary targets for white-box analysis.

The testing objective is white-box verification, evaluated primarily across two automatically meas urable structural coverage criteria: statement coverage and branch coverage, both instrumented via coverage.py (Python) and Jest (JavaScript). Additionally, the manual test cases in Chapter 2 were designed using basis path coverage — a structured path-enumeration methodology based on CFG construction and cyclomatic complexity — which provides systematic path-level guarantees that a utomated tooling does not measure.

The methodology adopts a dual-path design: manual analysis constructs human-written test cases as an interpretable, traceable baseline; LLM-assisted generation is introduced in parallel as an AI-powered alternative. The experiment further extends this into a multi-dimensional comparison fra mework — four LLMs (DeepSeek Chat, DeepSeek Reasoner, Gemini-2.5-Flash, and GPT-4o-min i) are evaluated under identical coverage criteria, enabling two levels of systematic comparison: h uman vs. AI, and AI vs. AI across models. Together, these comparisons form the central thread ru nning through the entire report.

2. Manual Test Case Design

This chapter presents the manual white-box test case design process applied to both subject progra ms: **convert_number_to_words.py** (Python) and **dayjs index.js** (JavaScript). The manual analysi s serves as a baseline for comparison with the AI-generated test cases described in Chapter 3. All t est cases were designed following a systematic methodology described below, before any AI tool was consulted.

2.1 Methodology

Manual white-box test design followed a three-step process applied consistently to every function under test:

Step 1 — Control Flow Graph (CFG) Construction.

The source code of each function was read line by line and annotated into basic blocks (B1, B2, …). Each block represents a maximal sequence of statements with no branching. Decision nodes (if/switch/match statements) and their edges were identified and drawn as a directed graph.

Step 2 — Cyclomatic Complexity Calculation.

Cyclomatic complexity V(G) was computed using the simplified formula :

V(G)=number of decision nodes+1. This value gives a lower bound on the number of linearly independent test paths required to achieve full branch coverage.

Step 3 — Independent Path Derivation and Test Case Design.

Starting from the CFG, a basis set of independent paths was enumerated — each path exercising at least one edge not covered by any previous path. One test case was designed per path, with input values chosen to force execution through that specific path and expected outputs determined by manual tracing of the code logic.

This process was applied to 6 functions in total across the two subject programs (3 per program). The resulting test cases were then executed using Jest (JavaScript) and pytest (Python) with coverage instrumentation.

2.2 convert_number_to_words (Python)

The Python module **convert_number_to_words.py** provides utilities for converting integers to their English word representations. Three functions were selected for manual analysis.

2.2.1 NumberingSystem.max_value

This class method returns the maximum integer supported by a given numbering system (SHORT, LONG, or INDIAN), implemented via a Python match-case statement.

Cyclomatic complexity:

1. V(G)=3+1=4 (three decision nodes: SHORT, LONG, INDIAN).
2. Four independent paths were identified and tested:

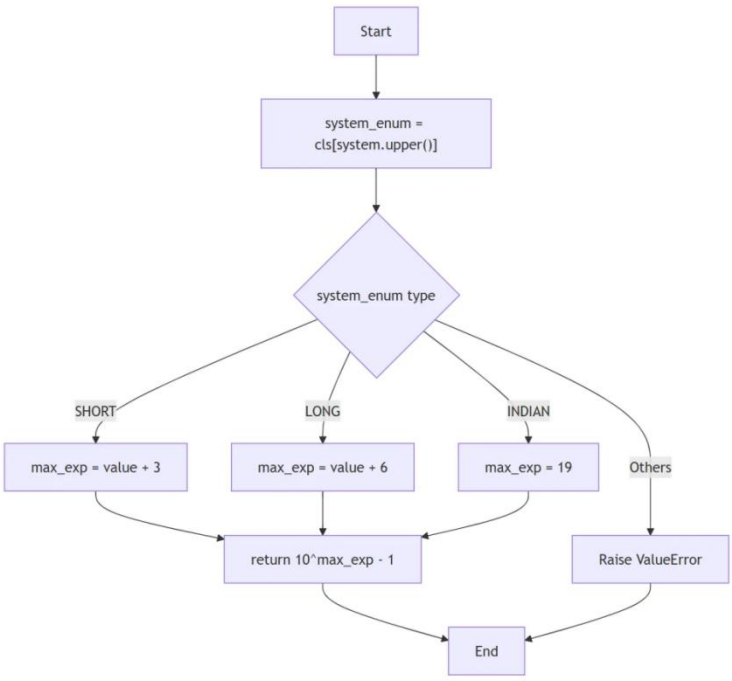

Figure 1 Control flow graph of NumberingSystem.max_value

Test ID	Input	Covered Path	Expected Output	Notes

CNTW-001	"short"	Path 1: SHORT branch	10^{18}-1	Normal case
CNTW-002	"long"	Path 2: LONG branch	10^{21}-1	Normal case
CNTW-003	"indian"	Path 3: INDIAN branch	10^{19}-1	Normal case
CNTW-004	"abc"	Path 4: invalid input	KeyError	Unreachable code finding(see 2.2.1)

Table 1 Test cases for NumberingSystem.max_value

Key finding — unreachable code:

Path 4 (invalid system name) was designed to trigger the case _: raise ValueError branch. However, manual tracing of the Python match-case semantics revealed that the assignment expression cls[system.upper()] raises a KeyError before the match statement is evaluated whenever an invalid system name is provided. Consequently, the case _ branch is structurally unreachable dead code. This finding was confirmed by running CNTW-004, which raised KeyError rather than ValueError. This type of semantic-level discovery — requiring close reading of language specification — illustrates a limitation that AI-generated test cases may overlook (discussed further in Section 4.4).

2.2.2 convert_small_number

This function converts an integer in [0, 99] to its English word form. It handles four distinct cases: negative input, input ≥ 100, single-digit numbers, teen numbers, and multiples of ten with optional ones.

Cyclomatic complexity:

1. V(G)=4+1=5 (decision nodes: num < 0, num ≥ 100, tens == 0, tens == 1).
2. Five independent paths were identified. CNTW-010 (42) and CNTW-011 (20) share the same basic block B10 but exercise different sub-branches (with and without hyphen), so both were included:

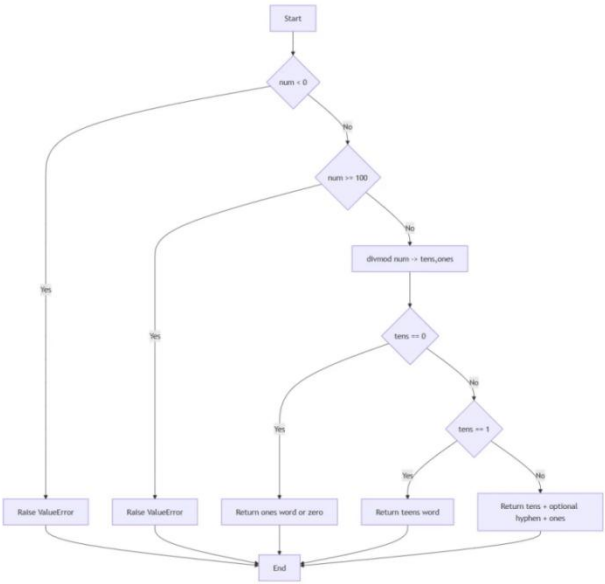

Figure 2 Control flow graph of convert_small_number

Test ID	Input	Covered Path	Expected Output	Notes
CNTW-005	-1	Path 1: num < 0	ValueError	
CNTW-006	100	Path 2: num >= 100	ValueError	
CNTW-007	0	Path 3: tens == 0	"zero"	
CNTW-008	7	Path 3: tens == 0	"seven"	
CNTW-009	15	Path 4: tens == 1	"fifteen"	
CNTW-010	42	Path 5: tens >= 2, ones != 0	"forty-two"	With hyphen
CNTW-011	20	Path 5: tens >= 2, ones == 0	"twenty"	No hyphen, same B10 block

Table 2 Test cases for convert_small_number

2.2.3 convert_number

This is the top-level recursive conversion function. It handles negative numbers, overflow detectio n, and recursive decomposition of large numbers into unit groups (thousand, million, etc.).

Cyclomatic complexity:

1. V(G)=5+1=6. The for-loop is not counted as a decision node because it iterates over a fixed-len gth enum structure (SHORT system: 6 groups), making its termination condition independent of r untime input.

2. Six independent paths were identified. CNTW-015 (345) is a compound path that simultaneousl y exercises Path 4 (recursive call for digit_group ≥ 100) and Path 5 (multi-unit concatenation). Thi s is an inherent property of recursive functions: the outer path necessarily executes the inner path. Both paths are counted as covered by a single test case, with a note that recursive path coverage re quires careful manual reasoning — a scenario where AI tools may produce incomplete cases:

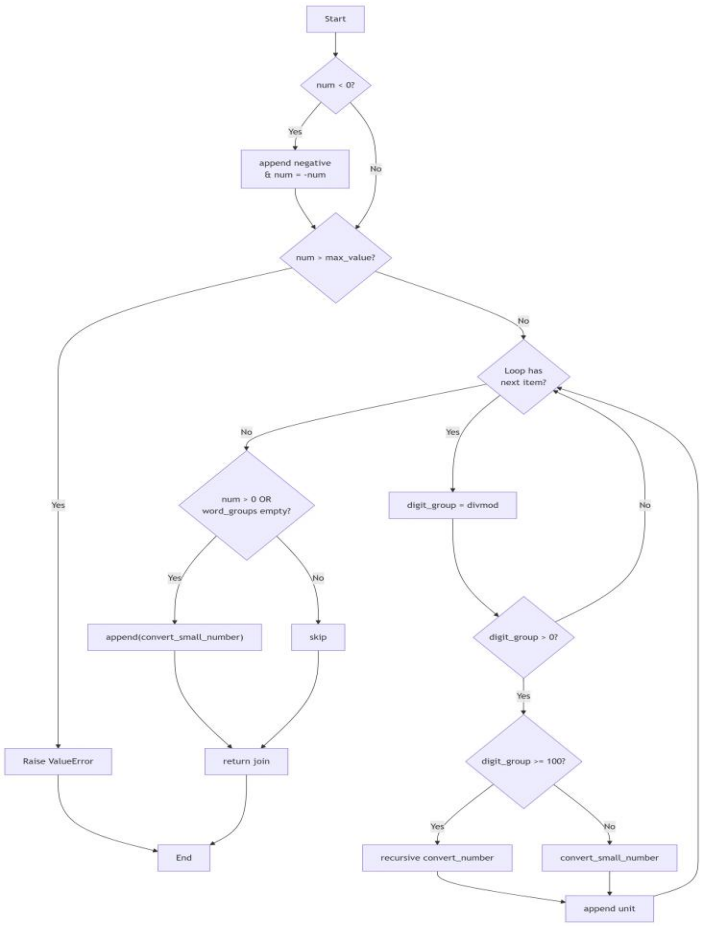

Figure 3 Control flow graph of convert_number

Test ID	Input	Covered Path	Expected Output	Key Branches Exercised
CNTW-012	-45	Path 1: negative	"negative forty-five"	num < 0 → True
CNTW-013	>max_value	Path 2: overflow	ValueError	num > max_value → True
CNTW-014	42	Path 3: small number	"forty-two"	digit_group > 0, digit_group < 100
CNTW-015	345	Path 4+5: recursion + multi-unit	"three hundred forty-five"	digit_group >= 100 → recursive call; compound path
CNTW-016	12345	Path 4+5: nested recursion	"twelve thousand three hundred forty-five"	Outer: thousand unit; inner: hundred unit recursion
CNTW-017	0	Path 6: zero / empty group	"zero"	word_groups empty → fallback convert_small_number

Table 3 Test cases for convert_number

2.3 dayjs (JavaScript)

Three internal functions from the dayjs source index.js were selected for manual analysis: parseDate, parseLocale, and diff. These functions represent a significantly more complex test subject than the Python module, due to their reliance on closures, global state, and the internal dayjs prototype chain.

2.3.1 parseDate

This function parses a variety of input types into a JavaScript Date object using a sequential chain of type guards.

Cyclomatic complexity:

1. V(G)=6+1=7 (decision nodes: null check, undefined check, instanceof Date, string && !Z, regex match, utc flag).
2. Paths 6 and 7 both reach return new Date(date), but through different decision branches (regex non-match vs. non-string input); they are non-equivalent under branch coverage:

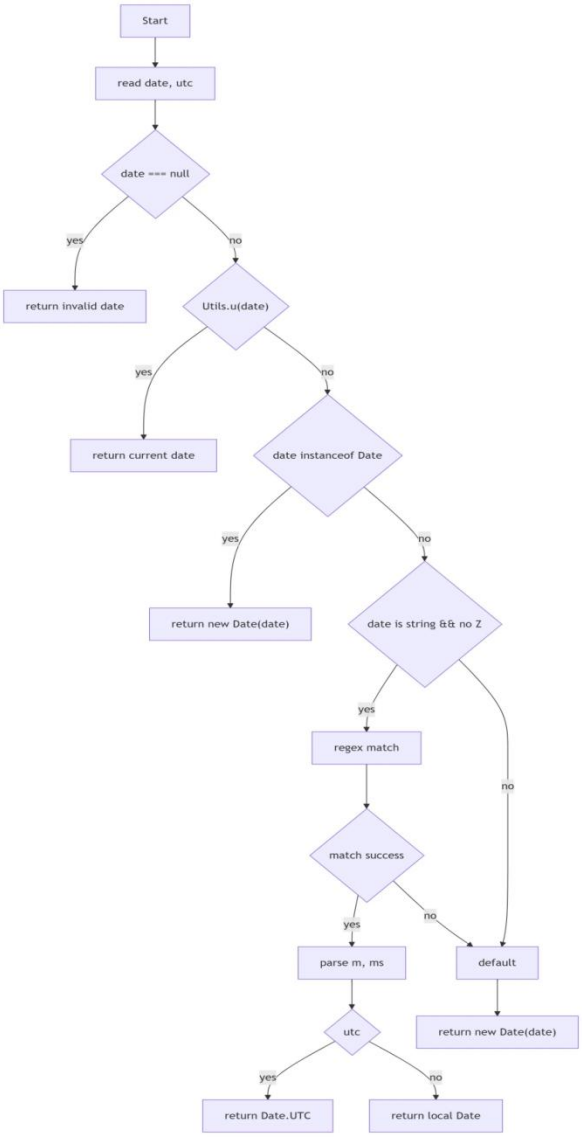

Figure 4 Control flow graph of parseDate

Test ID	Input	Covered Path	Expected Output	Notes
DJ-001	date = null	Path 1: null input	Invalid Date	
DJ-002	date = undefined	Path 2: undefined	Current datetime	
DJ-003	date = new Date()	Path 3: Date object	Copy of input Date	
DJ-004	"2023-01-15", utc=true	Path 4: string + regex match + utc=true	UTC Date object	
DJ-005	"2023-01-15", utc=false	Path 5: string + regex match + utc=false	Local Date object	
DJ-006	"invalid-date"	Path 6: string + regex fails	Invalid Date	Regex non-match fallback
DJ-007	1700000000000(timestamp)	Path 7: non-string fallback	Corresponding Date	Number input via new Date(date)

Table 4 Test cases for parseDate

2.3.2 parseLocale

This function resolves and optionally registers a locale configuration, with fallback logic for hyphenated locale codes (e.g., 'en-US' falls back to 'en').

1. Cyclomatic complexity:

V(G)=6+1=7 (decision nodes: !preset, string check, Ls[presetLower] exists, object provided, !l && split.length > 1, !isLocal && l).

2. Key design decision — DJ-010 vs DJ-012:

Both test cases use unknown locale strings, but they exercise different paths. **DJ-010** uses 'abc' (no hyphen): presetSplit.length === 1, so the fallback recursion condition is false and the function silently returns the current default locale. **DJ-012** uses 'en-US' (with hyphen): presetSplit.length === 2, triggering the recursive parseLocale('en') fallback. An AI model might classify both inputs as 'unknown locale' and generate only one test case, missing the structural difference introduced by the hyphen-split branch.

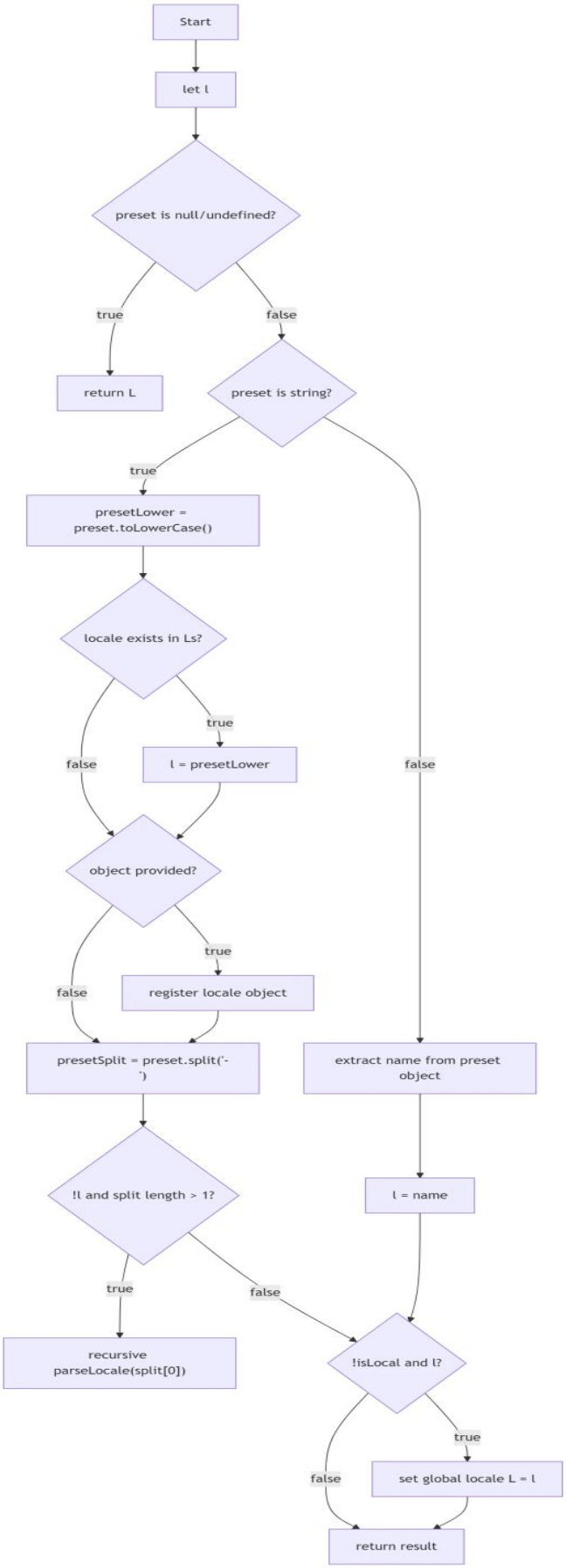

Figure 5 Control flow graph of parseLocale

Test ID	Input	Covered Path	Expected Output	Notes
DJ-008	parseLocale(null)	Path 1: preset empty	Return default locale L	
DJ-009	parseLocale("en")	Path 2: string, locale exists	Return 'en', update global L	
DJ-010	parseLocale("abc")	Path 3: string, locale not found, no hyphen	Return current default locale	AI likely merges DJ-010 and DJ-012 as same class
DJ-011	parseLocale("fr", frObj)	Path 4: string + object registration	Ls['fr'] created, return 'fr'	
DJ-012	parseLocale("en-US")	Path 5: fallback via hyphen split	Return 'en' via recursive call	Distinct from DJ-010: triggers fallback recursion
DJ-013	parseLocale({name:"jp", ...})	Path 6: object-form locale	Register 'jp', return 'jp'	
DJ-014	parseLocale("en", null, true)	Path 7: isLocal = true	Return 'en', global L unchanged	isLocal flag prevents global update

Table 5 Test cases for parseLocale

2.3.3 diff

This function calculates the time difference between two dayjs objects, with support for nine unit types (year, month, quarter, week, day, hour, minute, second, millisecond) and a float/integer output mode controlled by the third parameter.

Cyclomatic complexity:

V(G)=9+1=10 (nine switch branches, plus the float ternary operator adding one independent edge). All this = dayjs('2024-06-01T12:00:00').

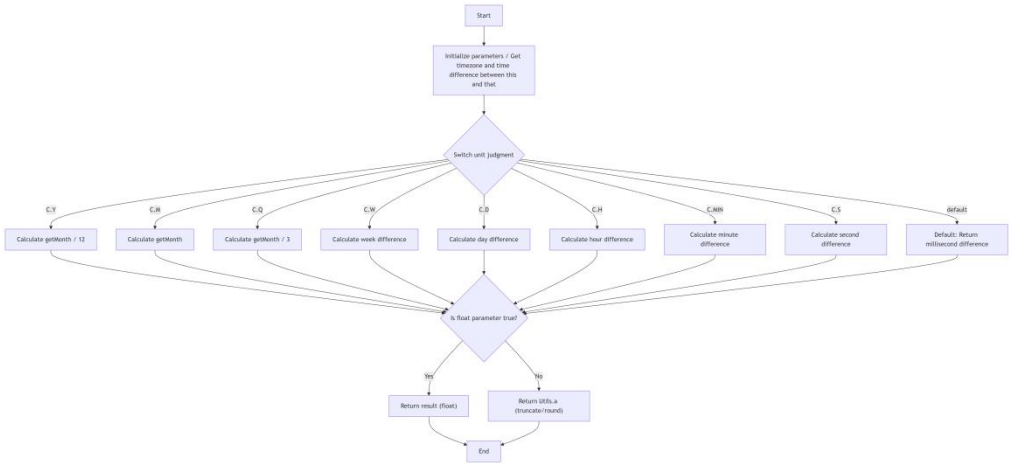

Figure 6 Control flow graph of diff

Test ID	Input	Path	Expected Output
DJ-015	('2023-06-01', 'year', true)	Path 1: Y + float	1
DJ-016	('2024-04-15', 'month', true)	Path 2: M + float	1.5
DJ-017	('2023-12-01', 'quarter', true)	Path 3: Q + float	2
DJ-018	('2024-05-18', 'week', true)	Path 4: W + float	2
DJ-019	('2024-05-30', 'day', true)	Path 5: D + float	2
DJ-020	('2024-06-01T09:30:00', 'hour', true)	Path 6: H + float	2.5
DJ-021	('2024-06-01T11:45:00', 'minute', true)	Path 7: MIN + float	15
DJ-022	('2024-06-01T11:59:30', 'second', true)	Path 8: S + float	30
DJ-023	('2024-06-01T11:59:59.500', 'ms', true)	Path 9: default + float	500
DJ-024	('2024-04-15', 'month', false)	Path 10: float=false	1
DJ-025	('2024-03-01', 'month', false)	Path 10: float=false	2
DJ-026	('2024-03-01', 'month', true)	Path 10: float=true (control)	1.548...
DJ-027	('2024-02-01', 'month', false)	Path 10: float=false	1

Table 6 Test cases for diff

DJ-024 (float = false)

The result of this test case is an integer. Under round, floor, or truncation semantics, the outcome is identical.

Therefore, this case alone cannot be used to distinguish the exact rounding behavior implemented in Utils.a().

DJ-025 and DJ-026 (paired test cases)

DJ-026 (float = true) is used to confirm the underlying floating-point value (approximately 1.548), establishing the ground truth.

Based on this value, DJ-025 (float = false) is used to verify the direction of integer truncation.

3. AI-based Test Generation Tool

3.1 Tool Architecture

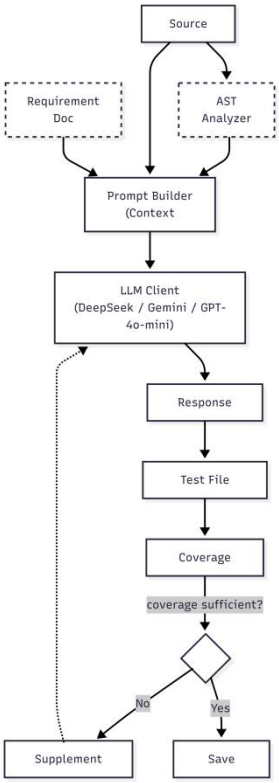

Figure 7 Tool architecture diagram of the AI-based test generation tool

The tool is implemented as a Python CLI application (main.py) that orchestrates a closed-loop pip eline across seven core modules. The overall workflow is as follows:

The closed-loop mechanism tracks the best coverage version across all rounds. If a supplemented script causes coverage regression or syntax errors, the system automatically rolls back to the previ ous best version.

3.2 Prompt Design

Five prompt types are used across different stages of the pipeline:

#	Prompt Type	Function	When Used
1	Initial (General)	build_general_prompt	General
2	Initial Prompt (AST-assisted Mode)	build_python_prompt	Python targets with --ast flag.
3	Supplement (JSON)	build_supplement_prom pt	Closed-loop rounds in JSON mode.
4	Script Generation	build_script_prompt	--script mode, initial generation.
5	Fragment Suppleme nt	build_supplement_script _prompt	--script mode, closed-loop rounds.

Table 7 Prompt design overview (five prompt types used across pipeline stages)

A key engineering decision in Prompt 5 is the fragment concatenation strategy: rather than regener ating the full script (which risks overwriting passing tests), only new test method bodies are gener

ated and spliced before the closing }); of the existing file. Syntax validation (node --check / ast.parse) is applied immediately after each concatenation.

Full prompt templates are provided in Appendix A.

3.3 Supported LLM Backends

The tool supports four LLM backends through a unified LLMClient interface:

Backend	Model	API Protocol	Notes
DeepSeek	deepseek-chat	OpenAI-compatible	Balanced accuracy and speed.
DeepSeek	deepseek-reasoner	OpenAI-compatible	Highest coverage on complex targets; slowest (up to 670s for Dayjs).
Google	gemini-2.5-flash	Google GenerativeAI SDK	High volume generation; accuracy varies.
GitHub	gpt-4o-mini	OpenAI-compatible (GitHub)	Fastest (~30s); lower accuracy on complex targets.

Table 8 Supported LLM backends and their invocation methods

DeepSeek and GitHub backends share the OpenAI-compatible client.chat.completions.create path. Gemini uses the google generative AI SDK with system prompt prepended to the user message, as it does not support the system role in the same way. All backends use temperature=0.3 to reduce output randomness.

3.4 Three Core Design Features

3.4.1 Multi-LLM Comparative Experimentation (--compare mode)

The --compare flag sequentially invokes all configured LLM backends against the same source file under identical conditions, then automatically generates a structured comparison report covering accuracy, statement coverage, branch coverage, and runtime. This enables objective, data-driven model selection and is the basis for the AI-vs-AI analysis in Chapter 4.

3.4.2 AST-assisted Structural Analysis (--ast flag)

For Python targets, the PythonAnalyzer module uses Python's built-in ast module to extract: function signatures and line ranges, all branch points (if/for/while/try), compound boolean conditions (and/or sub-expressions), and per-function execution paths via recursive CFG traversal. This structured information is injected into the prompt, replacing the LLM's self-analysis. The result: first-round accuracy improved from ~69% to ~95%.

3.4.3 Adaptive Output Mode (--script flag)

JSON format is fundamentally unable to express chain-call syntax such as dayjs('2023-01-01').diff('2022-01-01', 'year'). The --script mode addresses this by having the LLM generate a complete, runnable test script directly, rather than an intermediate JSON structure. For closed-loop supplement

ation, only method fragments are generated and concatenated (with rollback on regression), avoidi ng full regeneration.

4. Experimental Analysis

4.1 Experimental Overview

4.1.1 Experimental Setup

Test Subjects:

1. Python Project: `convert_number_to_words.py` - A number conversion utility with clear structu re and well-defined functions
2. JavaScript Project: `dayjs` - A date manipulation library with more complex nested structures a nd prototype-based inheritance

LLM Models Tested:

deepseek-chat , deepseek-reasoner , gemini-2.5-flash , gpt-4o-mini

Evaluation Metrics:

1. Coverage Metrics: Statement coverage and branch coverage
2. Accuracy Metrics: Test pass rate and test failure rate
3. Test Quality: Syntax correctness, logical validity, and framework compatibility

4.1.2 Key Findings Overview

1. Python Superiority: Python projects consistently achieved higher coverage (85.9%-91.0%) and accuracy (53.33%-95.83%) compared to JavaScript projects
2. Model Performance: gemini-2.5-flash emerged as the best performer for Python projects, while deepseek-chat showed the most consistent results for JavaScript
3. Common Error Patterns: Class method call errors were the most frequent issue in Python test ge neration, while syntax errors dominated JavaScript test generation
4. Generalization Challenges: The tool showed significant difficulty generalizing from Python to J avaScript, primarily due to differences in code structure and test framework complexity

4.2 False Negative Analysis

4.2.1 Python Project False Negative Analysis

Coverage Comparison

Test Type	Statement Coverage	Branch Coverage	Uncovered Lines
Manual Testing	90.0%	84.6%	51-52, 201-205
deepseek-chat	91.0%	92.3%	
deepseek-reasoner	91.0%	92.3%	
gemini-2.5-flash	91.0%	92.3%	
gpt-4o-mini (Best)	91.0%	92.3%	
gpt-4o-mini (Worst)	85.9%	84.6%	

Table 9 Coverage comparison between manual and AI-generated tests for Python project

Branch Coverage Calculation

The convert_number_to_words.py file contains 13 branches (including if statements and match-case branches). Manual testing covered 11 branches, missing 2 branches (the default branch in NumberingSystem.max_value method and the if __name__ == "__main__" branch), resulting in a branch coverage of 11/13 = 84.6%.

Uncovered Code Analysis

1. Lines 51-52 (NumberingSystem.max_value default branch): This branch handles invalid numbering system inputs, but manual tests did not cover this case.
2. Lines 201-205 (Main program doctest section): This code runs only when the module is executed directly, not when imported for testing, so it's expected to be uncovered.

4.2.2 JavaScript Project False Negative Analysis (Generalization Test)

Coverage Comparison

Test Type	Statement Coverage	Branch Coverage	Accuracy
Manual Testing	62.18%	49.45%	29.63%
deepseek-chat (Best)	76.5%	63.7%	100.00%
deepseek-chat (Worst)	0.0%	0.0%	0.00%
deepseek-reasoner (Best)	97.1%	85.2%	62.22%
deepseek-reasoner (Worst)	28.1%	20.3%	0.00%
gemini-2.5-flash (Best)	58.4%	44.0%	0.36%
gemini-2.5-flash (Worst)	31.1%	18.1%	0.00%
gpt-4o-mini (Best)	34.0%	20.9%	9.76%
gpt-4o-mini (Worst)	26.5%	18.7%	0.00%

Table 10 Coverage comparison between manual and AI-generated tests for JavaScript project

Manual Test Execution Results

1. Total test cases: 27
2. Passed test cases: 19
3. Failed test cases: 8
4. Test pass rate: 70.37% (19/27)
5. Test accuracy: 29.63% (8/27, considering only completely correct test cases)

Uncovered Code Analysis

JavaScript projects show significantly lower coverage and stability compared to Python projects.

Many code lines and branches remain uncovered, including: 17-205, 404, 416-427, 449-453, 461, etc.

4.3 False Positive Analysis

4.3.1 Python Project False Positive Analysis

Model Accuracy Analysis

Model	Accuracy Range	Average Accuracy	Typical Performance
deepseek-chat	69.23% - 95.83%	87.29%	Stable, most rounds >90%
deepseek-reasoner	59.26% - 95.00%	84.18%	More variable, some poor rounds
gemini-2.5-flash	88.46% - 90.70%	89.45%	Most stable, highest accuracy
gpt-4o-mini	53.33% - 90.91%	75.01%	Most variable, lowest minimum

Table 11 Model accuracy analysis for Python project

Common False Positive Types

1. Class Method Call Errors

Issue: Generated test code often calls class methods as regular functions

Example: `max_value("short")` instead of `NumberingSystem.max_value("short")`

Impact: Test failure due to undefined function errors

2. Exception Handling Test Errors

Issue: Incorrect implementation of exception handling tests

Example: Improper use of pytest.raises or incorrect exception types

Impact: Tests that fail to properly validate error conditions

3. Expected Value Calculation Errors

Issue: Generated expected values sometimes don't match actual code output

Example: Incorrectly calculated large number conversions

Impact: Test failures despite correct code behavior

4.3.2 JavaScript Project False Positive Analysis (Generalization Test)

Model Accuracy Analysis

Model	Accuracy Range	Average Accuracy	Typical Performance
deepseek-chat	0% - 100%	47.01%	Highly variable, many 0% rounds
deepseek-reasoner	0% - 62.22%	13.26%	Mostly low accuracy
gemini-2.5-flash	0% - 0.36%	0.09%	Almost complete failure
gpt-4o-mini	0% - 9.76%	1.95%	Very poor performance

Table 12 Model accuracy analysis for JavaScript project

Common False Positive Types

1. Test Case Format Errors

Issue: Generated JSON test cases often have invalid syntax

Example: Missing fields, incorrect data types, or malformed structure

Impact: Test cases cannot be parsed or executed

2. Syntax Errors in Test Code

Issue: Generated JavaScript test code contains numerous syntax errors

Example: Undefined variables, incorrect function calls, or missing brackets

Impact: Tests fail to compile or run

3. Test Framework Usage Errors

Issue: Incorrect use of Jest testing framework syntax

Example: Improper nesting of describe/test blocks or incorrect expect usage

Impact: Tests fail due to framework-specific errors

4.4 Model Comparison Analysis

4.4.1 Python Project Model Ranking

1. gemini-2.5-flash: Average accuracy 89.45%, stable coverage, best overall performance
2. deepseek-chat: Average accuracy 87.29%, stable coverage, good performance
3. deepseek-reasoner: Average accuracy 84.18%, more variable performance
4. gpt-4o-mini: Average accuracy 75.01%, most variable performance, lowest minimum

4.4.2 JavaScript Project Model Ranking

1. deepseek-chat: Average accuracy 47.01%, highly variable but occasionally good
2. deepseek-reasoner: Average accuracy 13.26%, occasional good performance
3. gpt-4o-mini: Average accuracy 1.95%, poor performance
4. gemini-2.5-flash: Average accuracy 0.09%, almost complete failure

4.4.3 Model Characteristics

Model	Strengths	Weaknesses	Application Scenarios	Average Generation Speed
deepseek-chat	Stable performance on Python projectsFast test case generation- Occasional good JavaScript results	Highly variable on JavaScript- Limited understanding of complex code structures	Suitable for Python projects and simple JavaScript code	100-400 seconds
deepseek-reasoner	High accuracy on Python projects (up to 95%)- Good coverage on JavaScript in some rounds	Slow generation speed- Variable performance	Suitable for complex Python projects requiring deep reasoning	400-700 seconds
gemini-2.5-fl	Most stable on Python projects- Highest accuracy- Ge	Very poor JavaScript performance- S	Highly recommended for Python project	150-500 seconds

ash	nerates comprehensive test cases	lower generation	s, not recommended for JavaScript	
gpt-4o-mini	Fastest generation speed- Lowest cost (as a mini model)	Most variable performance- Poor JavaScript performance	Suitable for quick prototyping or simple Python projects	20-60 seconds

Table 13 Model characteristics comparison

4.5 Language Difference Analysis

4.5.1 Python vs JavaScript Performance

Aspect	Python Projects	JavaScript Projects
Coverage Range	85.9%-91.0%	0%-97.1% (mostly <50%)
Accuracy Range	53.33%-95.83% (mostly >80%)	0%-100% (mostly <10%)
Test Code Quality	Good, few syntax errors	Poor, many syntax errors
Stability	High	Low

Table 14 Python vs. JavaScript performance comparison

4.5.2 Language-Specific Challenges

Python Code Structure

Simple structure with clear functions and minimal complexity

Straightforward syntax and semantics

Well-supported by LLMs due to extensive training data

JavaScript Code Structure

Complex structure with nested functions, closures, and prototype methods

Flexible but complex syntax

ES6+ features (import/export, arrow functions) less well understood by LLMs

Prototype-based inheritance presents significant challenges

4.5.3 Test Framework Differences

pytest (Python): Simple syntax, straightforward assertions, easy exception handling

Jest (JavaScript):More complex nested structure, diverse assertion methods, complex async testing

4.6 Root Cause Analysis

4.6.1 False Negative Root Causes

The main causes of false negatives (uncovered code and branches) are summarized as follows:

1.Limited Code Structure Understanding

LLMs may fail to fully capture complex control flow and hidden branches, especially in cases involving recursion or language-specific constructs. This leads to missing coverage in error handling paths and boundary conditions.

2.Test Generation Logic Deficiencies

Errors in transforming generated test cases into executable code (e.g., incorrect method calls or pa rameter usage) reduce the effectiveness of otherwise valid test intentions.

3.Ineffective Coverage Feedback Mechanism

The closed-loop feedback process does not always accurately target uncovered branches. As a resu lt, additional test cases may fail to improve coverage in subsequent rounds.

4.6.2 False Positive Root Causes

The main causes of false positives (incorrect or failing test cases) include:

1.Test Code Generation Errors

LLM-generated tests often contain syntax or logical mistakes, leading to failures unrelated to the a ctual program behavior.

2.Exception Handling Misunderstanding

Incorrect use of testing frameworks (e.g., improper exception assertions) results in invalid test vali dation.

3.Language-Specific Feature Misinterpretation

LLMs struggle with language-specific features such as Python's match-case or JavaScript's protot ype-based design, causing incorrect assumptions in test generation.

1. Enhanced Code Structure Information

4.7 Improvement Insights

To address the limitations identified in the experimental analysis, several key improvement strateg ies were derived from both experimental results and tool development experience.

1. Enhancing Code Understanding with AST Integration

Providing structured code information significantly improves test generation accuracy.

By incorporating AST-based analysis (e.g., function structure, branch points, and execution paths), the LLM can better understand program logic and generate more accurate and complete test cases.

This approach improved first-round accuracy from approximately 69% to over 95% in Python proj ects.

2. Prompt Optimization for Better Test Quality

Prompt design plays a critical role in guiding LLM behavior. Effective strategies include:

Explicitly specifying coverage goals (e.g., branch and boundary coverage)

Highlighting error-handling paths and edge cases

Providing correct examples of test structures (e.g., class method calls in Python)

These improvements reduce common issues such as incorrect assertions and invalid test structures.

3. Language-Specific Adaptation

Experimental results show that a one-size-fits-all approach is ineffective across programming lang uages.

Python benefits from simpler syntax and clearer structure

JavaScript requires special handling due to closures, prototype chains, and more complex test fram

eworks (e.g., Jest)

Therefore, language-specific templates and analysis strategies are necessary to improve generalization performance.

4. Improving Coverage Feedback Mechanism

Although the closed-loop generation process increases coverage, its effectiveness is limited by insufficient targeting of uncovered code.

Future improvements should focus on:

Precisely identifying uncovered branches

Generating targeted supplementary test cases

Avoiding redundant or irrelevant test generation

5. Strengthening Test Validation and Robustness

To reduce false positives, additional validation mechanisms are required:

Syntax checking before execution

Automatic correction of common errors (e.g., method call format)

Filtering invalid inputs that violate function preconditions

These measures improve the reliability and executability of generated test cases.

6. Model Selection and Practical Usage Strategy

Different LLMs exhibit distinct strengths:

High-accuracy models are suitable for complex reasoning tasks

Faster models are useful for rapid iteration

A practical approach is to select models based on task complexity and combine AI-generated tests with manual verification to ensure overall quality.

5. Project Report

This project has implemented an LLM-based automatic white-box test case generation tool. Operating as a Python CLI, it supports two target languages (Python/JavaScript) and is compatible with multiple LLM backends including DeepSeek, Google Gemini, and GitHub GPT-4o-mini. Its core functionalities include code analysis, test case generation, coverage measurement, closed-loop supplementation, and multi-LLM comparison. This report will analyze three aspects: comparison with traditional non-AI white-box testing techniques, AI limitations encountered in practice and corresponding tool improvement methods, and project summary with future outlook.

5.1 Comparison with Traditional Non-AI White-box Testing Technologies

5.1.1 Advantages and Limitations of Traditional Technologies

Traditional non-AI white-box testing techniques mainly include AST-based static generation tools, symbolic execution, fuzzing, and manual test design. Each approach has its focus, yet all exhibit significant limitations in practical application.

AST-based static generation tools (e.g., JUnit Generator, PyTest Generator) can automatically gen

erate test method skeletons based on code structure but only produce "empty shells"—test methods are created, while specific input data and assertions still require manual specification by testers. Taking the target module convert_number_to_words.py in this project as an example, such tools can generate a test method named test_convert_number but cannot infer whether the expected output for input 123 should be "one hundred twenty-three" or "一百二十三". Testers must still understand the business logic and manually complete the assertions, resulting in limited automation.

Symbolic execution techniques (e.g., KLEE) generate test cases covering extreme paths by solving path constraints, offering strong theoretical coverage capabilities. However, in practice, when faced with recursive functions such as convert_number, the number of paths grows exponentially with the number of input digits, easily leading to "path explosion" that exhausts memory or renders computation time unacceptable. For more complex libraries like Dayjs, symbolic execution struggles to handle internal date calculations and timezone conversions.

Fuzzing (e.g., AFL) excels in security testing by generating random inputs and monitoring for program anomalies. However, its objective is to discover crashes and memory leaks rather than achieving high statement or branch coverage. For convert_number, fuzzing might generate -9999999 and verify that the program does not crash, but it will not check whether the output is "negative nine million nine hundred ninety-nine thousand nine hundred ninety-nine", thus failing to guarantee functional correctness.

Manual test case design is the most reliable approach. In this project, we designed 17 test cases for convert_number based on requirement documentation, covering all independent paths with 100% accuracy. However, this approach incurs extremely high costs—designing test cases for a mere 52-line function required approximately one hour, while testing a 500+ line library like Dayjs would require several days. In the context of rapid iteration in modern software development, such substantial human investment is unsustainable.

5.1.2 Core Advantages of This Project

The LLM-based test generation tool effectively complements the aforementioned traditional approaches, with its core advantages manifested in the following aspects:

Dimension	Traditional Technique Pain Points	Response to this project
Assertive Filling	AST tools generate only test skeletons; assertions require manual filling	LLM directly generates complete input-output assertions, e.g., expect(convert_number(123)).toBe("one hundred twenty-three")
Path Coverage	Symbolic execution prone to path explosion	LLM combined with AST analysis achieves 91% statement coverage and 92.3% branch coverage in the first round
Business Semantics	Fuzzing cannot verify output correctness	LLM can incorporate requirement documentation to generate assertions conforming to business rule

		s
Efficiency	Manual design is time-consuming	Generation, execution, and closed-loop supplementation of 24 test cases total only 138 seconds

Table 15 Comparison of traditional non-AI white-box testing technique pain points and this project's AI-based responses

Multi-mode Adaptability: To address different code styles, this project designed two generation modes. For pure functional code like convert_number, the JSON test case mode is employed: the LLM outputs a JSON structure such as [{"input":123,"expected":"one hundred twenty-three"}], which the tool converts to pytest assertions. For chain-call libraries like Dayjs, the JSON mode completely fails—dayjs('2023-01-01').format('YYYY') cannot be expressed as {input:"2023-01-01", expected:"2023"}. Therefore, we introduced the --script mode, allowing the LLM to directly generate executable test script code, successfully increasing Dayjs statement coverage from 26.9% to 97.1%.

AST Integration for Enhanced Understanding: Simply sending source code text to the LLM may cause the model to overlook hidden branches. Taking the match-case structure in the max_value function as an example, plain text cannot clearly convey the specific logic of each branch. By extracting structured branch information through AST—SHORT: max_exp = value[0][0] + 3, LONG: max_exp = value[0][0] + 6, INDIAN: max_exp = 19, _: raise ValueError—and injecting it into the prompt, the LLM-generated test cases precisely cover all four paths.

Closed-loop Feedback Mechanism: Traditional techniques generate test cases in a "one-shot" manner; if coverage targets are not met, manual supplementation is required. This project implements an automated closed loop: run tests →collect uncovered lines and branches →send uncovered code fragments to the LLM →generate supplementary test cases. For Python projects, first-round coverage reaches over 90%; for Dayjs projects, after 2-3 rounds of closed-loop iteration, coverage improves from approximately 30% to between 70% and 97%.

5.1.3 Limitations of this project

Despite its numerous advantages, this project is subject to constraints inherent to LLMs, resulting in the following limitations:

LLM Hallucination: In the Dayjs project, some models generated assertions such as expect(...).toBe(1.5), while the actual computed result was 1.5645.... The LLM assumed, based on common sense, that the month difference should be a neat 1.5, ignoring complex factors such as leap years and varying month lengths. Such hallucinations are less frequent in pure functional projects but are difficult to eliminate in scenarios involving dates, floating-point operations, and similar complexities.

Result Randomness: Running the same model on the same code multiple times may yield different numbers of generated test cases and varying accuracy rates. For instance, DeepSeek Reasoner's accuracy on convert_number ranged from 86.96% to 95.00% across multiple experiments. Traditional techniques do not exhibit such uncertainty.

Significant Performance Variance: Response times across different LLMs vary dramatically. G PT-4o-mini generates Python test cases in just 32.7 seconds, while DeepSeek Reasoner requires 2 71.5 seconds—a difference exceeding eightfold. For the Dayjs project, Reasoner's runtime reache d 670 seconds, severely impacting iteration efficiency.

Closed-loop Overfitting: During the supplementary generation phase, LLMs sometimes become overly focused on covering a single uncovered line, generating extreme or even invalid inputs that cause assertion failures. For example, to cover the tens == 1 branch in convert_small_number, the LLM generated num=100. However, this input is intercepted at the function entry by if num >= 10 0 and raises an exception, never reaching the target branch.

5.2 AI Limitations Encountered in Practice and Improvement Proces s

Throughout the project development and experimentation process, we encountered a series of prac tical problems stemming from LLM characteristics. These issues span multiple dimensions, includ ing code structure understanding, output format stability, and contextual integrity. The following s ections detail the specific AI limitations we encountered and the corresponding improvement proc esses.

5.2.1 Expressiveness Limitations of the JSON Format: LLMs Writin g Test Scripts Directly

In the early stages of the project, we designed a JSON-based test case format, expecting to unifor mly describe all testing scenarios using the "input-output" paradigm. While this design performed well for pure functional Python code, it exposed fundamental deficiencies when applied to chain-c all libraries such as Dayjs. A typical Dayjs testing scenario requires expression as:

However, the JSON format can only express {"input": "2023-01-01", "expected": "2023-02-01"}, completely losing the intermediate method call chain. The essence of the problem lies in the fact t hat chain calls are not suited to the "input→output" abstraction, and JSON as an intermediate form at has inherent expressiveness limitations.

Therefore, we re-evaluated the tool's design abstraction and decided to abandon the intermediate f ormat, instead having the LLM directly output executable test scripts. This led to the development of the --script mode, incorporating three key design elements:

Initial Generation: The prompt includes import statements and basic structure examples for the te sting framework, requiring the LLM to output complete test file code.

Closed-loop Supplementation: To avoid truncation risks associated with full script regeneration, we adopted a "fragment concatenation" strategy—the LLM generates only new test method fragm ents, which the program automatically appends before the closing }); of the existing script. Concur rently, the prompt sends only uncovered lines and their surrounding three lines of context, signific antly reducing input length.

Quality Assurance: Immediately after concatenation, syntax validation is performed (node --chec k for JavaScript). If syntax errors are detected, the supplement is discarded and the previous versio

n retained. A rollback mechanism ensures the highest-coverage script version is always preserved. After these improvements, DeepSeek Chat's statement coverage increased from 26.9% to 72.7%, while DeepSeek Reasoner reached 97.1%; accuracy improved from 0% to 97.37%.

5.2.2 Coverage Gaps in AST Analysis: The Analysis Module Requires Continuous Evolution

While analyzing the max_value function in convert_number_to_words.py, we discovered that the AST analysis results showed an empty "branch points" list, despite the actual code containing a match-case structure with four distinct branch paths. Investigation revealed that the PythonAnalyzer module initially only handled four statement types—if, for, while, and try—and completely ignored the match statement introduced in Python 3.10.

This occurred because the Python language encompasses dozens of statement types, while JavaScript has even more. Our initial assumption—that core control flow consists only of if, for, and while statements—overlooked the reality of language evolution. With the widespread adoption of Python 3.10, match-case structures are increasingly used, and such omissions result in incomplete code structure information provided to the LLM, adversely affecting test case generation quality.

A corresponding solution is to extend the PythonAnalyzer with a visit_Match method that traverses node.cases to extract each branch's pattern and body, generating structured descriptions.

This also prompted us to reconsider the role of AST analysis. Attempting to exhaustively enumerate all syntax structures is impractical. A more sustainable strategy is to at least record the line numbers and types of unknown nodes, prompting the LLM to be aware of code blocks requiring attention. Additionally, AST analysis should complement source code text—the AST provides precise structural information, while source code offers complete semantic context.

5.2.3 The Context Window Trade-off: Why Providing Only Local Code Leads to Unreasonable Test Cases

In the closed-loop supplementation phase, to reduce prompt length and response latency, we sometimes provided the LLM with only the uncovered line and a small amount of surrounding code context. This design was based on the notion that "local problems should be solved locally," but it exposed problems in practice.

For example, in one experiment, the uncovered line was return NumberWords.TEENS.value[ones] inside the convert_small_number function. The model was given only a narrow local snippet, without enough information about the function signature, entry constraints, and the dependency logic behind tens and ones. Consequently, the LLM generated convert_small_number(100) as input attempting to cover this branch. However, 100 is intercepted at the function entry by if num >= 100, raising an exception and never reaching the target branch.

The fundamental cause was that context window compression compromised logical integrity. For a branch within a function, the LLM requires the following information:

1. Function signature (input type and value range)
2. Preceding validity checks (which inputs are intercepted early)

3. Dependency relationships for the current branch (the origin of tens and ones)

Providing only local code snippets prevents the LLM from acquiring this critical information. This suggests that a more robust context strategy should include the following:

1. Providing richer function-level context when the uncovered line appears inside a function
2. Including relevant call-site context when branch reachability depends on inter-function usage
3. Making entry constraints and validity checks more explicit in prompts
4. Filtering obviously invalid candidate inputs before they are reused in later testing stages

Therefore, context compression should not sacrifice logical integrity; for branch-oriented supplementation, richer functional context is often more reliable than a narrowly local snippet.

5.2.4 Conflict Between LLM "Common Sense" and Code "Facts": Logical Deviations in Floating Point and Rounding

In the Dayjs project, LLM-generated assertions exhibited two systematic error patterns:

1. **Floating-point precision**: for diff('2024-04-15', 'month', true), the LLM generated expect(result).toBe(1.5), while the actual return value was 1.564516129032258.
2. **Rounding Rules**: for diff('2024-03-01', 'month', false), the LLM expected 2 (intuitive rounding), while the actual return value was 1 (flooring).

Both problems stem from the same underlying issue—the LLM relies on "common sense" rather than "code facts." Common sense suggests that from January 15 to March 1 is approximately 1.5 months, which rounds to 2. However, Dayjs's diff implementation is based on day count conversion, influenced by leap years and month lengths, and the integer mode uses Math.floor rather than rounding. The LLM learned general concepts of time differences from its training data but did not grasp the specific implementation details of Dayjs, leading to systematic biases.

To mitigate this issue, three practical strategies can be considered:

Prompt refinement: one possible improvement is to explicitly describe the library's floating-point and flooring behavior in the prompt.

Assertion strategy adjustment: for floating-point results, assertion styles such as toBeCloseTo() are often more appropriate than strict equality.

Human-review support: another useful improvement would be to flag suspicious strict-equality assertions involving time differences for manual inspection.

In later Dayjs experiments, accuracy reached 97.37% after broader workflow optimization, although this improvement should not be attributed to any single isolated mechanism.

5.2.5 Uncontrollable Output Format: Multi-Layer JSON Parsing for Fault Tolerance

Despite repeated emphasis in prompts to "return only JSON, without any additional text," LLMs frequently add explanatory text, code block markers, or even output incomplete JSON. Typical error patterns:

"Here are the test cases:\njson\n[{...}]\n"

"I have generated the following test cases:\n[\n {...}\n]\n"

"\n[\n {...}\n]\n" (without json identifier)

"{...}\n{...}" (multiple individual objects, not an array)

"[{\"id\":\"TC001\", ..." (truncated, unclosed brackets)

LLMs are trained on large corpora containing numerous examples formatted as "question + explanation + answer." The models naturally gravitate toward conversational output patterns. Even when instructed to "return only JSON," the model may interpret "return only" as meaning "the final output should contain only JSON," treating preceding explanations as also part of the "output."

Therefore, we abandoned the expectation of complete reliance on the LLM adhering to format constraints and instead implemented a multi-layer parsing function that attempts parsing in sequence:

Direct json.loads(response)

Extract ```json ... ``` code blocks via regex

Extract [...] patterns without code block markers

Extract { ... } patterns (attempting to repair unclosed brackets for truncated cases)

Scan line by line, extracting individual JSON objects and concatenating into an array

This function comprises approximately 100 lines of code but significantly improved parsing success rates. JSON parsing success rate increased from approximately 70% to near 100%.

5.2.6 Closed-loop Supplementation Stability: Rollback Mechanisms and Syntax Validation

During multi-round closed-loop supplementation, we encountered two stability-impacting issues:

Coverage Regression: Round 1 coverage 71%, Round 2 86%, Round 3—the newly generated script had syntax errors causing Jest execution failure, coverage dropping to 0%.

Syntax Errors: LLM-generated test fragments, after concatenation, frequently exhibited issues such as mismatched parentheses, unclosed quotes, or chain-call errors.

These two issues both point to a core contradiction—balancing the need to allow LLMs freedom to explore new paths for coverage improvement against the need to preserve existing achievements. In attempting to cover more code, LLMs may "overreach," compromising the integrity of existing scripts. Therefore, we constructed a "best version tracking + syntax validation" mechanism:

Best Version Tracking: Record script content and coverage after each round. If new coverage > historical best, update the best version; if new coverage < historical best, discard the current round's content and roll back to the best version.

Syntax Validation: Immediately after concatenation, perform syntax checks (using node --check for JavaScript, ast.parse for Python). If syntax errors are detected, discard the supplement, retain the previous version, and log the error information.

Error Feedback: If a supplement is discarded due to syntax errors, add a note to the next round's prompt: "The code generated in the previous round had syntax errors; please correct them."

After this, cases of "coverage dropping to zero after multiple supplementation rounds" no longer occurred. Script syntax correctness improved from 85% to nearly 100%.

5.2.7 Deduplication and Overfitting: Outstanding Issues Requiring Further Resolution

During multi-round closed-loop supplementation, the LLM may generate duplicate test cases. For instance, TC001 and TC042 both test convert_small_number(5), differing only in descriptive text. This increases test execution time without improving coverage. An effective deduplication mechanism has not yet been implemented, primarily because the LLM generates inputs in diverse forms (integers, floats, strings, etc.), making simple hash-based deduplication inadequate.

Additionally, despite prompt and context optimization, some experiments still exhibit cases where the LLM generates extreme inputs to cover a specific line of code. For example, to cover NumberWords.TENS.value[tens], the LLM generated num=200, which falls outside the valid range for convert_small_number, causing test failure. This essentially stems from incomplete understanding of preconditions or missing context for code fragments.

Therefore, for deduplication, introducing a deduplication mechanism based on input parameter hashing could be considered, but parameter normalization (e.g., treating 5 and 5.0 as equivalent) must first be addressed. For overfitting, adding hard constraints in the prompt stating that "generated test cases must pass all validity checks at function entry" could be planned, complemented by code-level pre-filtering.

5.3 Experimental Data Comparison

The following table presents comparative results before and after implementation of key improvements:

Improvement item	project	metric	Before improvement	Improved	change
script pattern	Dayjs	statement coverage	26.9%	72.7%	+45.8%
script pattern	Dayjs	precision	0%	97.37%	+97.37%
AST assist	Python	precision	69.23%	95.45%	+26.22%
Grammar check + rollback	be in common use	Script syntax accuracy rate	85%	99%+	+14%
Multi-level JSON parsing	be in common use	JSON parsing success rate	70%	99%+	+29%

Table 16 Experimental data comparison before and after key improvements

5.4 Project Summary and Outlook

5.4.1 Core Project Outcomes

This project successfully constructed an LLM-based automatic white-box test case generation too

l, realizing full-process automation spanning code analysis, test case generation, test execution, co verage measurement, and closed-loop supplementation. Key achievements are manifested in the fo llowing areas:

Complete Module Development: Seven core modules were built, including code analysis, LLM i nvocation, test case generation, coverage measurement, experiment tracking, and CLI entry point, realizing end-to-end automated white-box test case generation workflow with support for Python/J avaScript target languages and multiple LLM backends.

Key Technological Innovations:

1. **Adaptive Output Mode (--script)**: To address testing difficulties with JavaScript chain-call lib raries, we designed both JSON test case and direct script generation modes, increasing Dayjs state ment coverage from below 30% to above 97%.
2. **AST-integrated Analysis**: By extracting code structure information through the Python AST m odule and injecting it into prompts, first-round test case quality improved significantly, achieving coverage targets efficiently.
3. **Multi-LLM Comparative Experimentation**: The --compare parameter enables automatic sequ ential execution across all configured LLM backends, generating comparison reports covering cov erage, accuracy, and runtime metrics, providing data support for model selection.
4. **Stable Closed-loop Feedback Mechanism**: An automated "generation-execution-analysis-sup plementation" closed-loop process was established. Through engineering approaches including fra gment concatenation, rollback mechanisms, syntax validation, and multi-layer JSON parsing, issu es such as LLM output truncation, coverage regression, and format instability were effectively res olved, resulting in final output scripts with syntax correctness exceeding 99% and JSON parsing s uccess approaching 100%.
5. **Multi-dimensional Experimental Validation**: Dozens of experiments were conducted across t wo projects (convert_number_to_words.py and index.js), covering scenarios including presence/a bsence of requirement documentation, AST assistance, --script mode, and multi-model compariso ns, with over 30 experimental data records collected. Experimental results validated the tool's effe ctiveness and revealed performance patterns of different LLMs across various code styles.

5.4.2 Project Experience Summary

Through the development and experimentation process of this project, we accumulated several pra ctical experiences regarding AI-assisted software testing:

1. **Abstraction Design Must Align with Code Style**: The JSON format is suitable for describing pure functional code but exposes expressiveness limitations when facing chain calls. When existin g abstractions cannot adequately represent the target problem domain, reconstructing the abstracti on is preferable to patching within the old framework. Having the LLM directly write test scripts e ssentially acknowledges that "code itself is the most precise test description language."
2. **Context Integrity Takes Priority Over Length Optimization**: While compressing context du ring closed-loop supplementation can reduce prompt length and response latency, compromising l

ogical integrity prevents the LLM from correctly understanding code reachability. For branches within functions, richer function-level context is often more reliable than narrowly local snippets; for cross-function dependencies, additional call-chain context may also be beneficial.

3. **Engineering Fault Tolerance Is More Reliable Than Format Constraints**: Regardless of how strongly prompts emphasize "return only JSON," LLMs may still add explanations or output malformed formats. Rather than attempting to alter model behavior, constructing multi-layer parsing and fault tolerance mechanisms at the code level is more effective. The four-layer JSON parsing, syntax validation, and rollback mechanisms in this project exemplify this philosophy.

4. **Preserving the Best Historical Version Is Essential**: LLMs exploring new paths may compromise existing achievements; thus, maintaining the best version state is necessary. Rollback mechanisms ensure that even if a particular round's generation quality declines, the final output does not regress.

5. **AST Analysis Requires Continuous Evolution**: Language syntax continually evolves (e.g., Python's match-case), and analyzers must expand accordingly; otherwise, structural information omissions will occur. A more sustainable strategy is to design generic node handling logic that records at least location information for unknown nodes for LLM reference.

5.4.3 Future Development Directions

Based on the achievements and limitations of this project, combined with trends in LLM technology evolution, future optimization and expansion can proceed in the following directions:

1. **Enhance AST Analysis Capabilities**: Extend the Python analyzer to support additional syntax structures including async/with, try-except-else, and more; implement JavaScript AST analysis modules to enable structured information injection for JS projects; explore AST-based path constraint generation to complement LLM capabilities.

2. **Integrate Traditional Testing Techniques**: Combine LLMs with symbolic execution and fuzzing. Leverage symbolic execution to solve extreme boundary constraints and generate test cases covering complex conditions; leverage LLMs to generate test cases covering foundational paths and business rules, achieving complementary strengths and enhancing testing comprehensiveness.

3. **Expand Multi-language and Framework Support**: Add support for mainstream programming languages including Java, Go, and C++, along with adaptation to common testing frameworks such as JUnit, TestNG, and Google Test, improving the tool's general applicability.

4. **Implement Parallelization and Performance Optimization**: Introduce multi-threaded/distributed execution mechanisms to parallelize calls to multiple LLM backends and test case execution, significantly reducing total runtime. Simultaneously implement test case deduplication (based on input parameter hashing) to reduce redundancy.

5. **Enhance Test Case Validity Verification**: Develop LLM-based test case validation modules to automatically detect whether generated test cases conform to code logic and business rules, filtering invalid or erroneous test cases. This can be combined with static analysis results to perform pre-judgment on anomalous inputs.

6. **Continuously Optimize Prompt Engineering**: Based on LLM limitations discovered during e xperimentation, design more refined prompt templates. Introduce few-shot learning by including r epresentative test case examples in prompts; introduce chain-of-thought (CoT) prompting to guide LLMs through step-by-step reasoning of code logic and test case design, reducing hallucinations.

5.4.4 Overall Project Conclusion

This project successfully demonstrated the feasibility and effectiveness of white-box test case auto matic generation technology based on large language models. By integrating AST analysis, requir ement documentation, and closed-loop feedback mechanisms with LLMs, the tool achieved perfor mance comparable to manual white-box testing on pure functional code (91% statement coverage, 92.3% branch coverage), and achieved breakthrough improvements on chain-call libraries (97% a ccuracy). The LLM limitations exposed during experimentation—including insufficient context un derstanding, unstable output formats, and logical biases—were effectively mitigated through engin eering approaches such as fragment concatenation, rollback mechanisms, multi-layer parsing, and syntax validation.

The core insight of this project is that the focus of AI testing tool development should not be on pu rsuing perfect one-shot outputs from LLMs, but on constructing engineering systems capable of to lerating imperfect outputs. As LLM technology continues to advance, particularly with improvem ents in code understanding and reasoning capabilities, LLM-based white-box test automatic gener ation tools will play increasingly important roles in software testing. The future deep integration o f traditional testing techniques with AI technologies is poised to drive software testing toward high er levels of automation and intelligence.

6. Appendix

Appendix A: Full Prompt Templates

1. Initial (General)

```Python
prompt = f"""You are a white-box testing expert.

## Source Code ({language})

```{language}

{source_code}

```

{req_section}

Task

1. **First analyze the code structure**: identify all functions, branches (if/else/switch), loops, exception handling, and compound conditions.

2. **Then generate test cases** that cover the following goals:

- Statement coverage: cover every executable line

- Branch coverage: cover both True and False outcomes of each branch

- Condition coverage: cover each sub-condition combination in compound expressions

- Path coverage: cover all feasible execution paths
```

```
## Output Format

Please return a strict JSON array only (note: JSON cannot contain programming expressions such as `
10**18`; write the full number instead, such as `1000000000000000000`):

```json

[

  {{

    "id": "TC001",

    "description": "Test description",

    "input": {{"parameter_name": value}},

    "expected_output": "Expected output",

    "covered_branches": ["Branch description"],

    "covered_paths": ["Path description"]

  }}

]

```

Please return JSON only, with no other text."""
```

2. Initial Prompt (AST-assisted Mode)

```Python
prompt = f"""You are a white-box testing expert. Please generate comprehensive white-box test cases
 based on the source code and the code structure analysis below.

## Source Code

```python

{source_code}

```

Code Structure Analysis (from AST)

Function List

{functions_str}

Branch Points (if/for/while/try)

{branches_str}

Compound Conditions (and/or expressions)

{conditions_str}

Execution Paths

{paths_str}

{req_section}

Task

Generate test cases to achieve the following coverage goals:

1. **Statement coverage**: cover every executable line

2. **Branch coverage**: cover both True and False outcomes of each branch

3. **Condition coverage**: cover every True/False combination of each sub-condition in a compound e
xpression

4. **Path coverage**: cover all feasible execution paths

Output Format
```

```
Please return a strict JSON array only (note: JSON cannot contain programming expressions such as `
10**18`; write the full number instead, such as `1000000000000000000`):

```json

[

  {{

    "id": "TC001",

    "description": "Test description (explain which branch/condition/path is covered)",

    "input": {{"parameter_name": value}},

    "expected_output": "Expected output",

    "covered_branches": ["L line number: condition → True/False"],

    "covered_paths": ["Path description"]

  }}

]
```

Please return JSON only, with no other text."""
```

3. Supplement (JSON)

```Python
prompt = f"""The previously generated test cases do not provide sufficient coverage. Please add sup
plementary test cases.

## Current Coverage

- Statement coverage: {coverage_report.get('statement_coverage', 0):.1f}%

- Branch coverage: {coverage_report.get('branch_coverage', 0):.1f}%

## Uncovered Code Lines

{list(uncovered)}

## Uncovered Branches

{branches_str}

## Source Code (with uncovered lines annotated)

```

{annotated_source}
```

## Number of Existing Test Cases: {len(existing_tests)}

## Task

Please generate **only supplementary test cases**, focusing specifically on the uncovered lines and
 branches above.

Do not duplicate existing test cases.

## Output Format

Please return a strict JSON array only:

```json

[

  {{

    "id": "TC_SUP_001",

    "description": "Supplementary test description",
```
```

```
    "input": {{"parameter_name": value}},
    "expected_output": "Expected output",
    "covered_branches": ["Target uncovered branch"],
    "covered_paths": ["Target uncovered path"]
  }}
]
```
Please return JSON only, with no other text."""
```

# 4. Script Generation

### JavaScript branch

```
Python
if language == "javascript":
    framework_guide = f"""## Test Framework: Jest
Import style
```javascript
const _{module_name}Module = require('../targets/{source_filename}');
const {module_name} = _{module_name}Module.default || _{module_name}Module;
```
Requirements
- Use a describe/test/expect structure
- Use chained calls correctly (for example, {module_name}('2023-01-01').format('YYYY'))
- Use expect(() => {{ ... }}).toThrow() for exception tests
- Return a complete runnable Jest test file"""
    code_block_lang = "javascript"
```

### Python branch

```
Python
if language == "python":
    framework_guide = f"""## Test Framework: pytest
Import style
```python
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..', 'targets'))
from {module_name} import *
```
Requirements
- Use a class + test_ method structure
- Use pytest.raises() for exception tests
- Return a complete runnable pytest test file"""
    code_block_lang = "python"
```

### Prompt body
```

```Python
prompt = f"""You are a test engineer. Please generate a complete, runnable test script from the so
urce code and JSON test cases below.

## Source Code Under Test ({language})

```{language}

{source_code}

```

JSON Test Cases

```json

{test_cases_json}

```

{framework_guide}

Important

- Infer the test intent from each test case's description and input/expected_output

- If expected_output in the JSON is descriptive text (for example, "the corresponding date object
"), infer the actual return value from the source code logic

- Ensure every test case is converted into an executable test method

- Ensure the code syntax is correct, with all parentheses and quotes properly closed

- Return code only, wrapped in ```{code_block_lang} ``` and do not include any other explanation t
ext"""
```

5. Fragment Supplement

JavaScript example

```JavaScript
test('Supplementary: test description', () => {{

  const result = {module_name}('2023-01-01').someMethod();

  expect(result).toBe(expectedValue);

}});
```

Python example

```Python
def test_sup_001(self):
    result = some_function(args)
    assert result == expected_value
```

Prompt body

```Python
prompt = f"""You are a test engineer. The current test coverage is insufficient, so please generat
e 5-15 supplementary test methods for the uncovered code.

## Current Coverage

- Statement coverage: {coverage_report.get('statement_coverage', 0):.1f}%

- Branch coverage: {coverage_report.get('branch_coverage', 0):.1f}%

## Uncovered Code Snippet

```

{snippet}
```
```

```
```

Uncovered Branches

{branches_str}

Requirements

- **Output only the test methods themselves**, without import, require, describe, class, or other scaffolding

- {format_hint}

- Ensure the code syntax is correct, with all parentheses and quotes properly closed

- Return code only, wrapped in ```{code_block_lang} ```"""
```

# Appendix B: Requirements Document

# 1. Python

```Markdown
# Requirements Document: convert_number_to_words.py
## Overview
Convert integers into their corresponding English word representation, supporting three numbering systems (short, long, and Indian).
## Module Structure
### 1. NumberingSystem (Enum Class)
Defines the units and magnitudes of the three numbering systems:
| System | Description | Maximum Value |
|--------|-------------|---------------|
| SHORT | Commonly used in English-speaking countries: thousand, million, billion, trillion, quadrillion | 10^18 - 1 |
| LONG | Used in some European countries: thousand, million, milliard, billiard | 10^21 - 1 |
| INDIAN | Used in India: thousand, lakh, crore, lakh crore, crore crore | 10^19 - 1 |
Method::
- max_value(system: str) -> int: returns the maximum supported value for the specified numbering system
### 2. NumberWords (Enum Class)
Stores the mapping tables for English number words:
- ONES: English words for 0-9 (0 maps to an empty string)
- TEENS: English words for 10-19
- TENS: English words for 20, 30, ..., 90
### 3. convert_small_number(num: int) -> str
Converts a non-negative integer from 0 to 99 into English words.
Input requirements:
- Must be a non-negative integer (num >= 0)
- Must be less than 100 (num < 100)
Conversion rules:
- 0 → "zero"
- 1-9 → directly look up the ONES table ("one", "two", ...)
- 10-19 → look up the TEENS table ("ten", "eleven", ...)
- 20-99 → combine TENS and ONES with a hyphen "-" (for example, "twenty-five")
- Exact tens do not use a trailing hyphen (for example, "twenty", not "twenty-")
Exceptions:
- num < 0 → raise ValueError("This function only accepts non-negative integers")
- num >= 100 → raise ValueError("This function only converts numbers less than 100")
### 4. convert_number(num: int, system: str = "short") -> str
Converts any integer into English words with support for a specified numbering system.
Input requirements:
- num must be an integer
- system must be one of "short" (default), "long", or "indian"
Conversion rules:
- Negative numbers → prefix with "negative", then convert the absolute value (for example, -100 → "negative one hundred")
- 0 → "zero"
- Numbers larger than the maximum supported by the selected system → raise ValueError("Input numbe
```
```

```
r is too large")
- Split the number from high to low according to the magnitudes defined in the numbering system an
d recursively convert each part
  - Digit groups >= 100 are converted recursively via convert_number
  - Digit groups < 100 are converted via convert_small_number
- Join the parts with spaces
Output examples:
- convert_number(0) → "zero"
- convert_number(100) → "one hundred"
- convert_number(-100) → "negative one hundred"
- convert_number(123456789) → "one hundred twenty-three million four hundred fifty-six thousand se
ven hundred eighty-nine"
## Boundary Conditions and Special Cases
1. Zero handling: when the input is 0, word_groups is empty, so the second condition in num > 0 or
 not word_groups is taken
2. Negative number handling: add "negative" first, then take the absolute value of num
3. Maximum-value boundary: a value exactly equal to max_value should convert normally; max_value +
 1 should raise an exception
4. Recursive calls: digit groups >= 100 trigger recursive calls to convert_number
5. Numbering system validation: if an invalid system string is passed, NumberingSystem[system.uppe
r()] raises a KeyError
6. Hyphen rule: 21 → "twenty-one" (with hyphen), 20 → "twenty" (without hyphen)
7. Teens special handling: 10-19 have dedicated words and are not formed as combinations such as "
ten-one"
```

2. JS

```Markdown
# Requirements Document: index.js (Day.js Core Module)
## Overview
Day.js is a lightweight date and time manipulation library that provides an API similar to Moment.
js.
This file is the core module of Day.js and contains the core functionality for date parsing, forma
tting, comparison, and calculation.
## Module Structure
### 1. Global State Management
- L: global default language (initial value: "en")
- Ls: loaded locale objects (keyed by language name, value is the locale configuration)
### 2. Helper Functions
#### isDayjs(d) -> boolean
Determines whether an object is a Dayjs instance.
- The check is performed via instanceof Dayjs or the d.$isDayjsObject flag
#### parseLocale(preset, object, isLocal) -> string
Parses and sets the locale.
- When preset is a string: tries to find a loaded locale pack, with support for automatic fallback
 from "zh-cn" to "zh"
- When preset is an object: registers it directly as a locale pack
- When isLocal is false: updates the global default locale
#### parseDate(cfg) -> Date
Parses various date inputs into a JavaScript Date object.
- null → new Date(NaN) (invalid date)
- undefined → new Date() (current time)
- Date object → clone it
- String → use the regular expression REGEX_PARSE to match the "YYYY-MM-DD HH:mm:ss.SSS" format
  - Supports both UTC mode and local mode
  - Only strings that do not end with "Z" are matched by the regex
- Others → passed to new Date() (for example, timestamps)
### 3. dayjs(date, c) Factory Function
The entry function for creating Dayjs instances.
- If date is already a Dayjs instance, return a clone of it
- Otherwise, create a new Dayjs instance
### 4. Dayjs Class
#### Construction and Initialization
- constructor(cfg): parses locale settings and calls parse initialization
```

```
- parse(cfg): parses the date via parseDate, then calls init
- init(): extracts year, month, day, weekday, hour, minute, second, and millisecond from the Date
object
#### Query Methods
- isValid(): determines whether the date is valid (by checking whether toString() is "Invalid Date
")
- isSame(that, units): checks whether it is the same as another date (up to the specified unit)
- isAfter(that, units): checks whether it is after another date
- isBefore(that, units): checks whether it is before another date
#### Get/Set Methods
- $g(input, get, set): general getter/setter proxy
- get(unit): gets the value of the specified unit (year/month/day/hour/minute/second/millisecond)
- set(string, int): sets the value of the specified unit (returns a new instance)
- $set(units, int): internal set method (modifies the current instance directly)
  - Month/year updates require special handling to prevent date overflow (for example, when settin
g January 31 to February, use min(31, 28))
#### Calculation Methods
- add(number, units): date addition
  - Month/year: set via set
  - Day/week: calculated by days
  - Hour/minute/second: calculated by milliseconds
- subtract(number, string): date subtraction (internally calls add with the sign reversed)
- diff(input, units, float): calculates the difference between two dates
  - Supports year/month/quarter/week/day/hour/minute/second/millisecond units
  - The float parameter controls whether a floating-point result is returned
  - Considers timezone differences (zoneDelta)
#### Range Methods
- startOf(units): gets the start time of a specified unit (for example, start of month, start of y
ear, Monday for week)
  - The start day of the week is determined by the locale's weekStart setting
- endOf(arg): gets the end time of a specified unit (for example, month end at 23:59:59.999)
- daysInMonth(): gets the number of days in the current month
#### Formatting Methods
- format(formatStr): outputs the date according to a format string
  - Supported format tokens: YY, YYYY, M, MM, MMM, MMMM, D, DD, d, dd, ddd, dddd, H, HH, h, hh, a,
 A, m, mm, s, ss, SSS, Z, ZZ
  - Invalid dates return locale.invalidDate or "Invalid Date"
  - Characters inside square brackets are output as-is (escaped)
#### Output Methods
- valueOf(): returns the millisecond timestamp
- unix(): returns the second-level timestamp
- toDate(): returns a native Date object
- toJSON(): returns an ISO string for valid dates, null for invalid dates
- toISOString(): returns an ISO 8601 string
- toString(): returns a UTC string
#### Locale-Related Methods
- $locale(): gets the current instance's locale configuration object
- locale(preset, object): switches locale (returns a new instance)
- clone(): clones the current instance
### 5. Prototype Method Registration
Shortcuts are registered on the Dayjs prototype in a loop:
- millisecond(), second(), minute(), hour(), day(), month(), year(), date()
- These methods act as getters when called without arguments and setters when called with argument
s
### 6. Static Methods
- dayjs.extend(plugin, option): registers a plugin (each plugin is installed only once)
- dayjs.locale(preset): sets the global locale
- dayjs.isDayjs(d): checks whether the input is a Dayjs instance
- dayjs.unix(timestamp): creates an instance from a second-level timestamp
## Boundary Conditions and Special Cases
1. Invalid date: dayjs(null) creates an invalid date, and isValid() returns false
2. Timezone handling: utcOffset() rounds timezone offsets to the nearest 15 minutes (fix for the F
F24 browser bug)
3. Month-end overflow: when setting the month and the day overflows (for example, setting February
 from the 31st), automatically use the last day of that month
```

```
4. Week start day: startOf('week') depends on the locale's weekStart setting (for example, Monday
in Chinese, Sunday in English)
5. String parsing: strings ending with "Z" do not go through regex matching; they are passed direc
tly to new Date()
6. Plugin deduplication: extend uses the plugin.$i flag to prevent repeated installation
7. Timezone correction in diff: timezone differences between the two dates are considered when cal
culating the difference
```

Appendix C: Experiment Output Table

Target	Model	Test Cases	Time	Statement Cov.	Branch Cov.	Rounds	Pass/Fail	Accuracy	Timestamp
convert_number_to_words.py	deepseek-chat	24	138.0s	91.0%	92.3%	1	23/1	95.83%	2026-03-23 15:42:15
convert_number_to_words.py	deepseek-chat	22	90.5s	91.0%	92.3%	1	21/1	95.45%	2026-03-22 22:34:27
convert_number_to_words.py	deepseek-reasoner	20	259.7s	91.0%	92.3%	1	19/1	95.00%	2026-03-22 22:38:47
convert_number_to_words.py	deepseek-reasoner	27	434.8s	91.0%	92.3%	1	25/2	92.59%	2026-03-23 15:49:30
convert_number_to_words.py	gpt-4o-mini	22	32.7s	91.0%	92.3%	1	20/2	90.91%	2026-03-22 23:04:53
convert_number_to_words.py	gemini-2.5-flash	44	151.4s	91.0%	92.3%	3	39/4	90.70%	2026-03-22 16:35:26
index.js	deepseek-reasoner	27	498.0s	97.1%	85.2%	2	28/17	62.22%	2026-03-23 14:01:17
index.js	deepseek-chat	96	580.8s	69.3%	62.6%	3	33/0	100.00%	2026-03-23 12:19:17
index.js	deepseek-chat	38	365.5s	72.7%	63.7%	3	37/1	97.37%	2026-03-23 12:49:17
index.js	deepseek-chat	32	334.5s	76.0%	63.2%	3	31/1	96.88%	2026-03-23 12:40:56
index.js	deepseek-chat	22	398.8s	64.7%	57.1%	3	19/3	86.36%	2026-03-23 13:06:18

Figure 8 Experimental output table (comparative results across models and projects)